

Generics: Number Object?

Warum erlaubt IntelliJ scheinbar ein Objekt vom Typ `Number`?

```
public class Main {
    static Number id = 1;

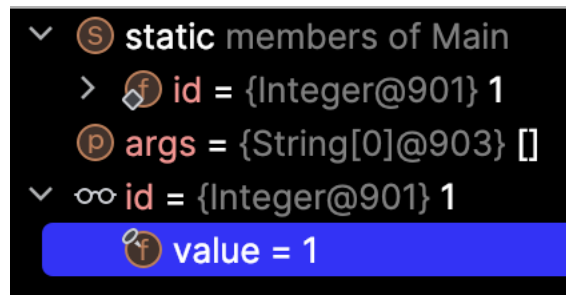
    static void main(String[] args){
        System.out.println("Das Objekt hat die Klasse: " + id.getClass());
    }
}
```

Der Print sagt allerdings schon einiges aus:

```
Das Objekt hat die Klasse: class java.lang.Integer
```

```
Process finished with exit code 0
```

Debug:



Die 1 ist ein primitiver `int`. Java weiß aber, dass `id` ein Objekt (Typ `Number`) erwartet.

Autoboxing: Der Compiler verpackt den primitiven `int` automatisch in seine Wrapper-Klasse `Integer`. Es entsteht also ein **konkretes Objekt** vom Typ `Integer`.

Jetzt haben wir ein `Integer`-Objekt. Da die Klasse `Integer` von der Klasse `Number` erbt (`extends Number`), gilt die "Ist-ein"-Beziehung: Ein `Integer` **ist eine** `Number`.

Deshalb darfst du eine `Integer`-Referenz in einer `Number`-Variable speichern.

Die Regel für abstrakte Klassen lautet: **Du kannst keine Instanz der abstrakten Klasse selbst erzeugen.**

Verboten: `new Number()` -> Du versuchst, das abstrakte Ding direkt zu bauen.

Erlaubt: `Number n = new Integer(1)` -> Du baust ein **konkretes** Ding (`Integer`), das zufällig auch eine `Number` ist.

Bei `Number id = 1;` wird im Speicher (Heap) **niemals** ein Objekt vom Typ `Number` angelegt. Es wird ein Objekt vom Typ `Integer` angelegt. Die Variable `id` ist nur das "Etikett", das auf dieses `Integer`-Objekt klebt.